

HACKATHON

SECURITY AND PRIVACY

Pompeu Fabra University
School of Engineering · 2nd semester 2025-26

CHALLENGE 1: AUCTION

Design and implement a smart contract on Sepolia that runs an auction.

CHALLENGE 2: ESCROW CONTRACT

Design and implement a smart contract that acts as an escrow between a buyer and a seller where:

- Neither party trusts the other.
- The buyer wants the product.
- The seller wants guaranteed payment.

CHALLENGE 3: VOTING SYSTEM

Design and implement a smart contract that runs a vote.

CHALLENGE 4: SOCIAL RECOVERY VAULT

A user stores their funds in a smart contract. If they lose access to their wallet, they should be able to recover their funds with the help of a group of trusted friends. Design and implement a smart contract on Sepolia that uses advanced on-chain cryptography to make this possible.

REMEMBER, SMART CONTRACTS ARE...

PUBLIC

IMMUTABLE

**TRIGGERED BY
A TRANSACTION**

DETERMINISTIC

RULES

IMPORTANT

The challenges we will give you are a starting point, **a way to get you thinking about the cryptographic tools and the design problems involved.** But we do not want you to just implement what is written on the handout and stop there.

The most interesting solutions will be the ones where you ask yourselves: **what is this actually useful for?** Who are the real actors? What could go wrong in practice? Maybe your escrow solves a problem you have actually faced, or your voting system is designed for a specific community you belong to.

In this hackathon, cryptography is a means to an end, and in this case, the end is building something that matters. **So, be creative, make it yours, and surprise us.**



RULES

1. You are encouraged to use AI tools (we recommend **Claude**, especially for Solidity code. However, use it wisely: let the thinking come from you. Do not ask AI to design your protocol from scratch, that is your job!
 2. We don't expect you to learn Solidity (nor master Ethereum). We want you to focus on the protocol design.
 3. Your solution must make meaningful use of cryptographic primitives from the course. The cryptography should be at the heart of your design, not an afterthought.
 4. Everyone in the group must understand the final protocol and be able to explain it. We will ask.
 5. You will impress us if you contextualise your solution in an original, creative scenario.
-

ADVICE

1. A partial solution that is well understood and clearly reasoned is worth more than a complete solution that nobody in the group can explain.
 2. Start with the simplest possible version that works, then add complexity.
 3. Do not panic, teachers are around to help you if you.
 4. Have fun! 🎉
-

GENERAL GUIDANCE

When designing your solution, consider questions such as:

1. What are you trying to protect? (privacy, fairness, funds, integrity, availability)
2. Who could try to break your system?
3. What can they observe on the blockchain?
4. What happens if a participant refuses to cooperate?
5. Can someone copy another participant's transaction?
6. Can someone learn secret information from small input spaces?
7. Can funds get stuck forever?

You are not required to solve every possible issue, but you must show that you have thought about them.

PRESENTATION & EVALUATION

DELIVERABLES

The day before the presentations (Monday 9th), we will require you to hand over:

- **Sepolia contract address** or **repository with code**
 - **Presentation slides**
 - **A README file (~1page)** explaining the following.
 - **Scenario.** What problem the system solves, who it is designed for, and why using a blockchain-based solution makes sense.
 - **Actors and assumptions.** Who the participants are, which actors are assumed to be honest or potentially malicious, and what information is publicly visible on the blockchain.
 - **Protocol.** A step-by-step explanation of how the system works, including the happy path (normal operation) and possible failure cases.
 - **Threats and attacks.** Describe at least one concrete attack, including its potential impact and how the system mitigates or prevents it.
 - **Primitives.** List the cryptographic primitives used and their security properties.
 - **How to reproduce the demo.** Provide 2–5 steps explaining how to reproduce the demo.
-

PRESENTATION

On the last seminar, you will **present your work** in front of your classmates, teachers, and some external jurors.

Each member of the group will have **1min to present (plus 1-2min for the demo)**. **Prepare well**, there will be a timer!

Use your time to **cover the following**:

- What is your scenario? Who are the actors, which of them are assumed to be honest, and which are potentially adversarial?
 - What is your protocol? Walk us through the full flow: what happens in the happy path, and what happens when something goes wrong?
 - What motivated your design? What attack or vulnerability did you identify, and how did your design address it?
 - What cryptographic primitives did you use and why? What security property does each one provide?
 - Demo. Show it working (live or recorded, both are fine).
 - Did you use AI? Where and how was it used?
-

EVALUATION

General presentation (20%)

- Clarity of the design.
- Ability to explain and defend the system.
- Scenario or context in which the solution is presented.
- Justified use of AI.

System design (20%)

- Are the actors and their roles clearly defined?
- Are the assumptions about who is honest and who is adversarial reasonable and well justified?
- Is the full protocol flow explained, including failure cases?

Threat model (20%)

- Is there at least one concrete attack identified?
- Is it clear how each design decision was motivated by a specific threat?
- Did they think about realistic attacks?

Cryptographic primitives (20%)

- Are the primitives used listed and explained?
- Is it clear what security property each one provides and why it was chosen?

Demo (20%)

- Does the contract work on Sepolia?
- Is the happy path demonstrated?
- Is at least one edge case or attack scenario shown?

Bonuses

- Particularly creative or original scenarios.
 - Deep solutions that show several design iterations.
 - Full working application (e.g. a webpage) that interacts with the smart contract.
 - Proposals for future work.
-

HAVE FUN!
